Capturas de pantalla

GameStore — Tienda de videojuegos con Bootstrap 5 y JavaScript

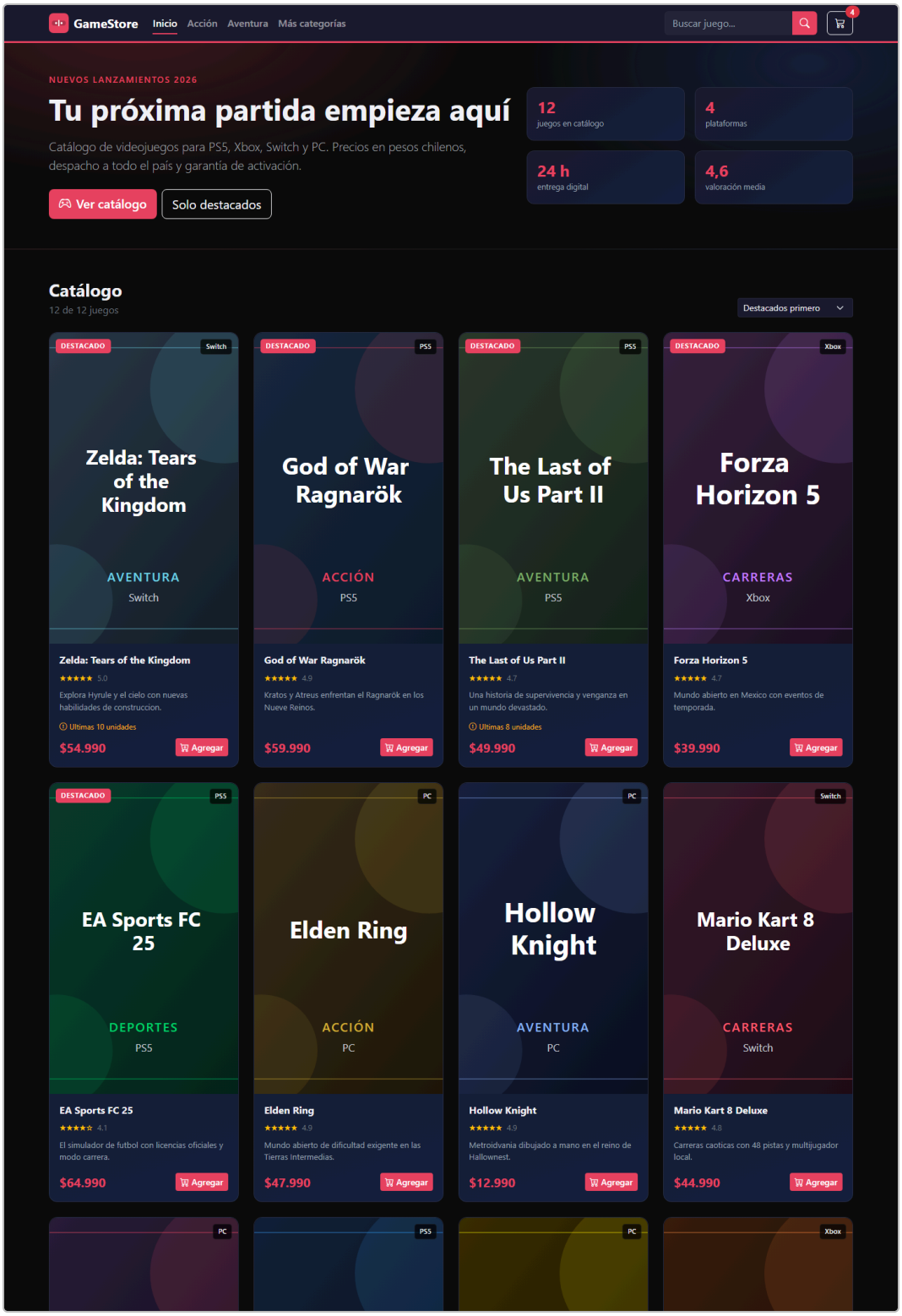
Alumno: Jean Santoni Camilo **Asignatura:** Desarrollo Frontend I (PFY2201)
Actividad: Optimizando la lógica y rendimiento de una página web con JavaScript — Semana 6
Repositorio: github.com/JeanSantoni02/frontend1
Sitio publicado: jeansantoni02.github.io/frontend1

1. Estructura de la página

Organización de archivos del proyecto:

```
index.html
assets/
    css/
        styles.css
    js/
        ui.js
        carrito.js
        productos.js
        app.js
    data/
        productos.json
    img/
        logo.svg
        god-of-war.svg
        zelda-totk.svg
        ... (12 portadas)
capturas/
```

La página se divide en cuatro bloques: barra de navegación fija, portada con los datos de la tienda, catálogo de productos y pie de página.



Página completa en escritorio: navbar, portada, catálogo y pie de página.

Diseño responsivo

La misma página en pantalla de móvil. El menú se agrupa en el botón hamburguesa y las tarjetas pasan a una sola columna.

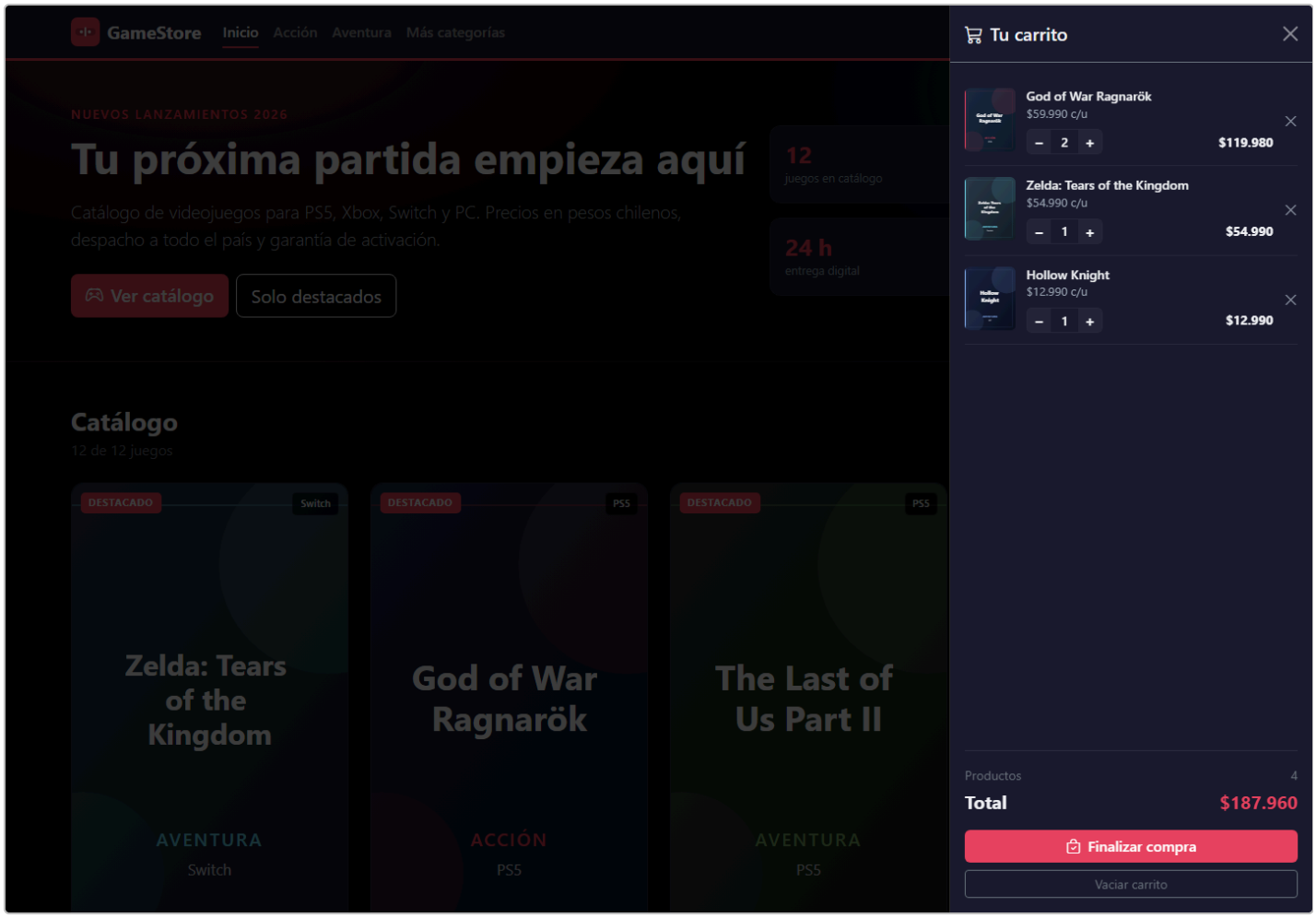


Vista en móvil (500 px de ancho).

2. Interacciones dinámicas

Agregar al carrito (evento click)

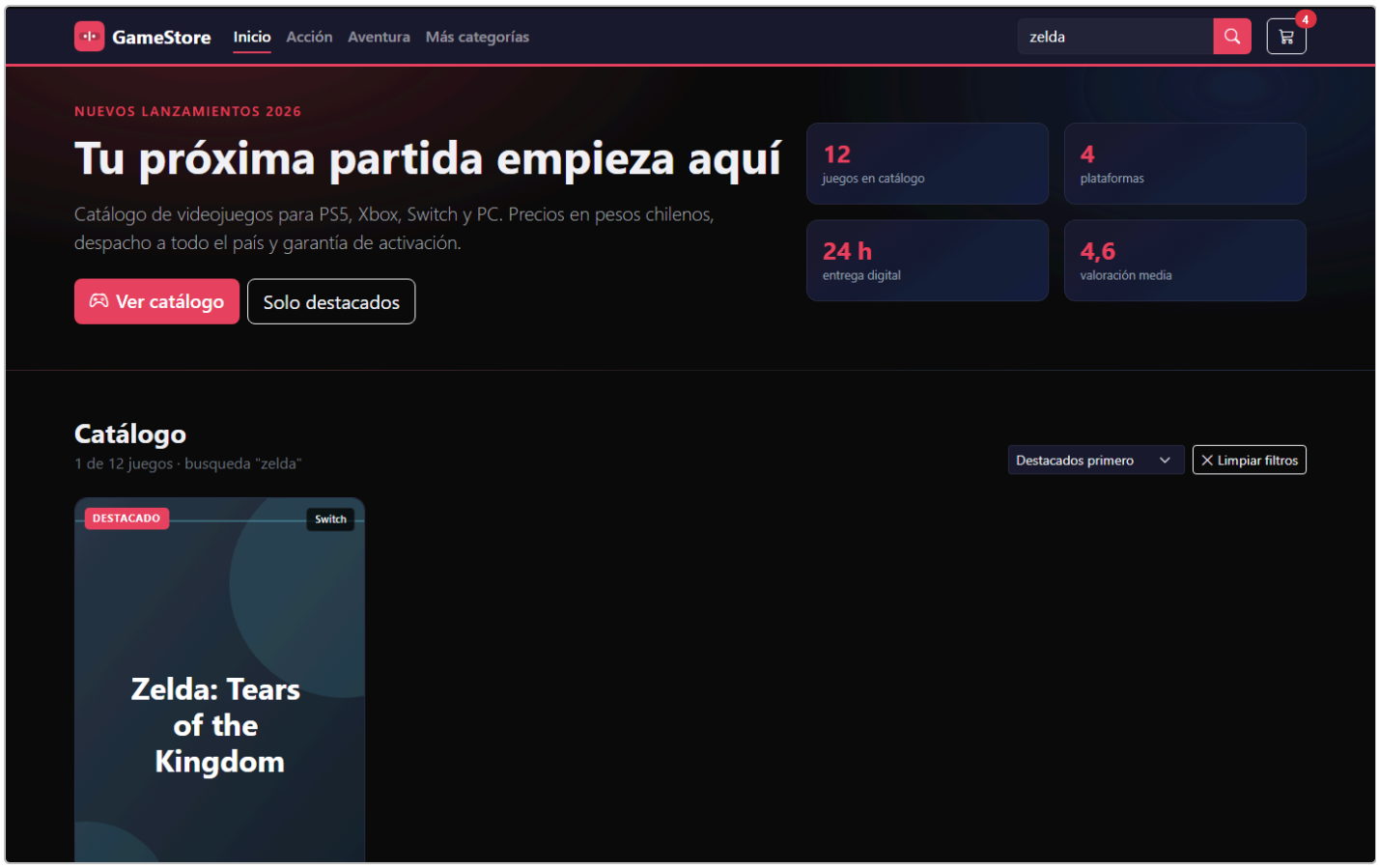
Al pulsar «Agregar» el producto entra al carrito, el contador de la barra se actualiza y el panel lateral muestra el detalle con cantidades, subtotales y total.



Carrito con 3 productos y 4 unidades. Total calculado: $187.960.

Búsqueda (evento submit)

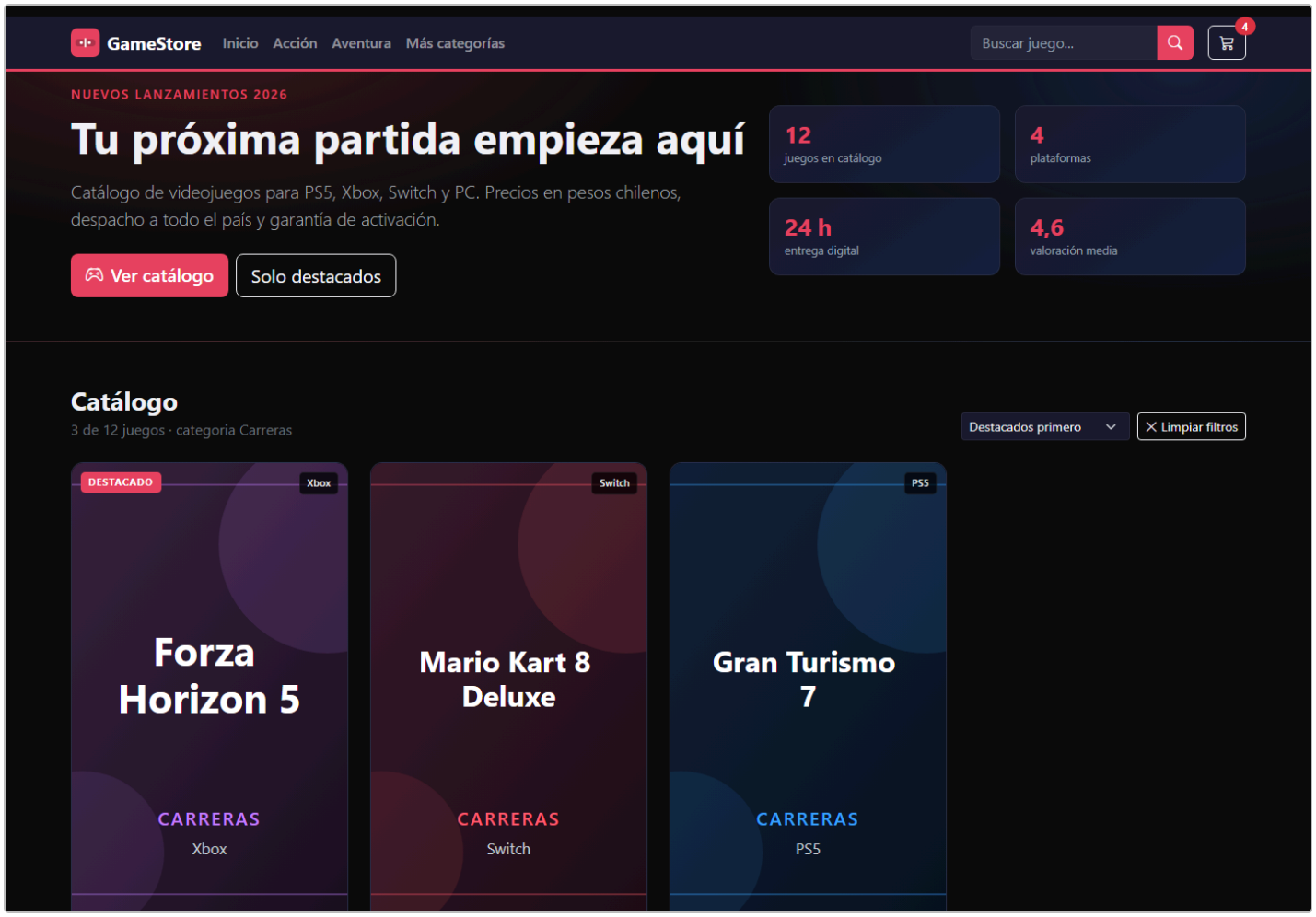
El formulario de la barra filtra el catálogo sin recargar la página. El texto bajo el título indica cuántos resultados quedaron.



Búsqueda de "zelda": 1 de 12 juegos.

Filtro por categoría

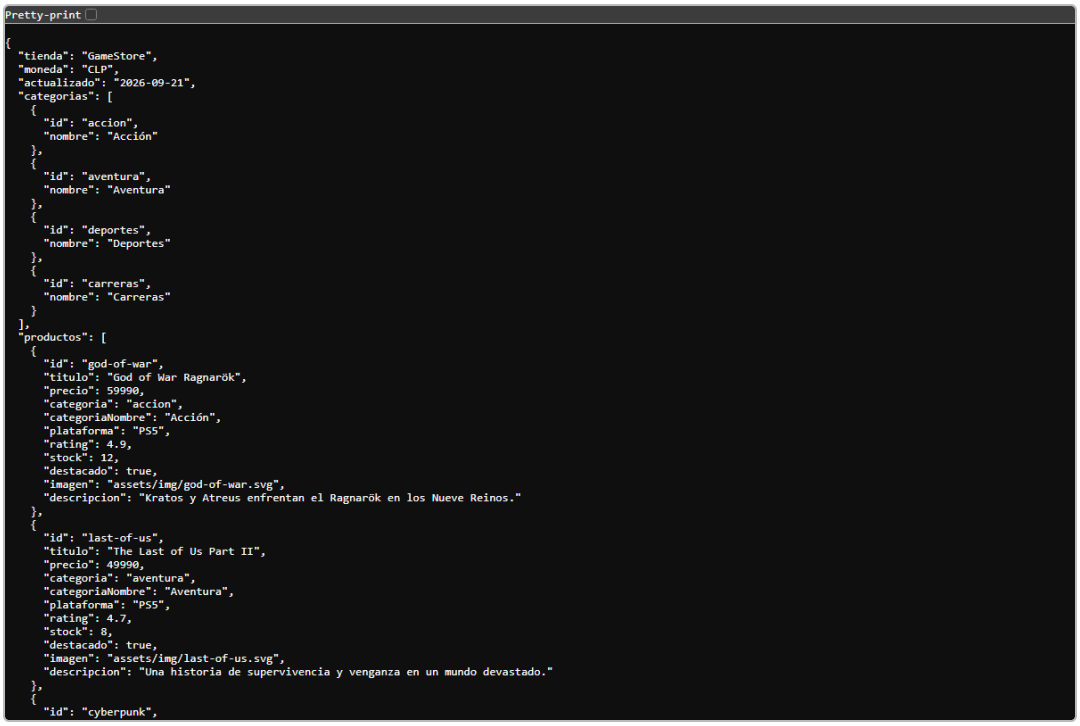
Los enlaces del menú filtran el catálogo y marcan la categoría activa.



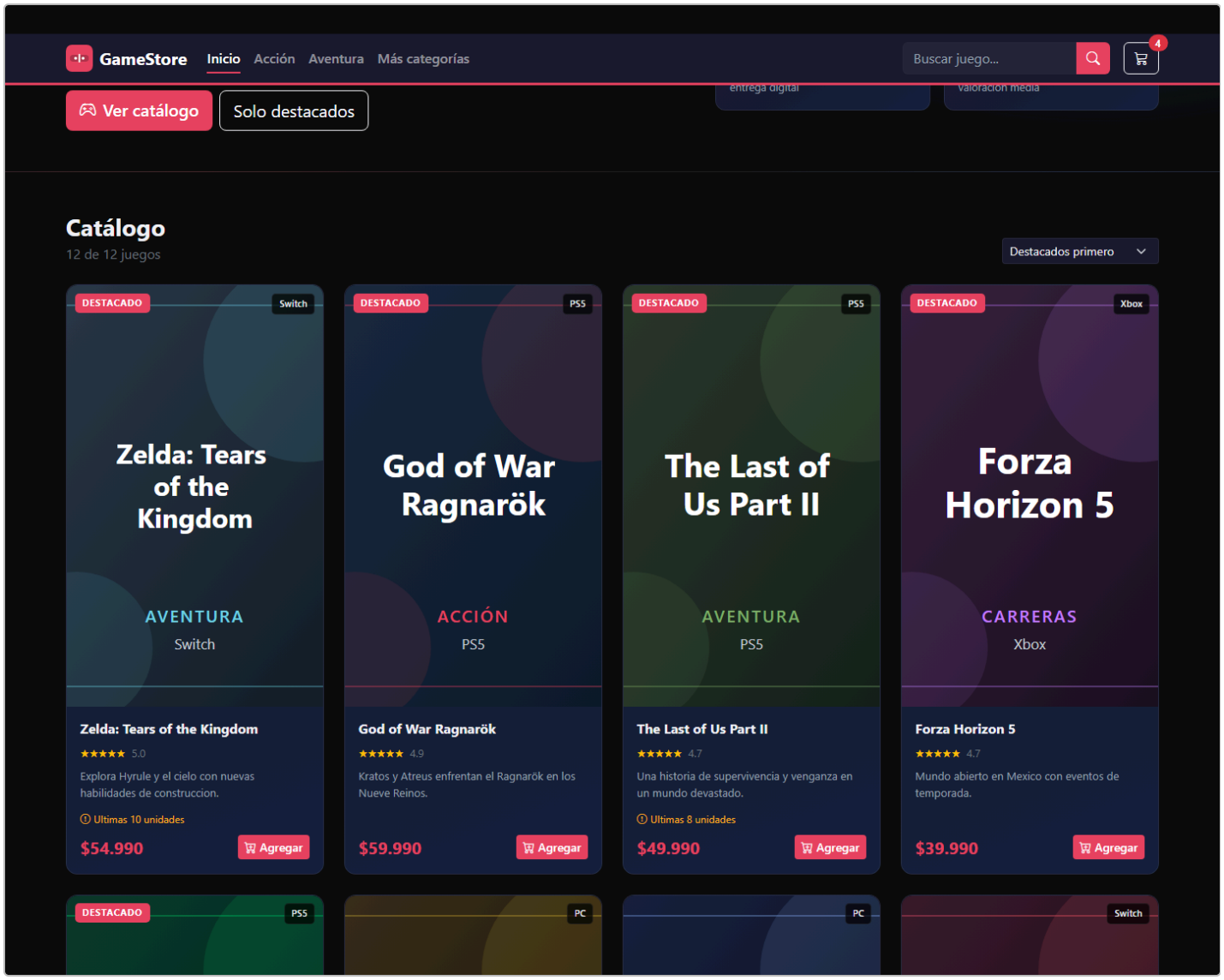
Categoría Carreras: 3 de 12 juegos.

3. Carga de datos externos con la Fetch API

Los productos no están escritos en el HTML: se descargan desde **assets/data/productos.json** y las tarjetas se generan con JavaScript.



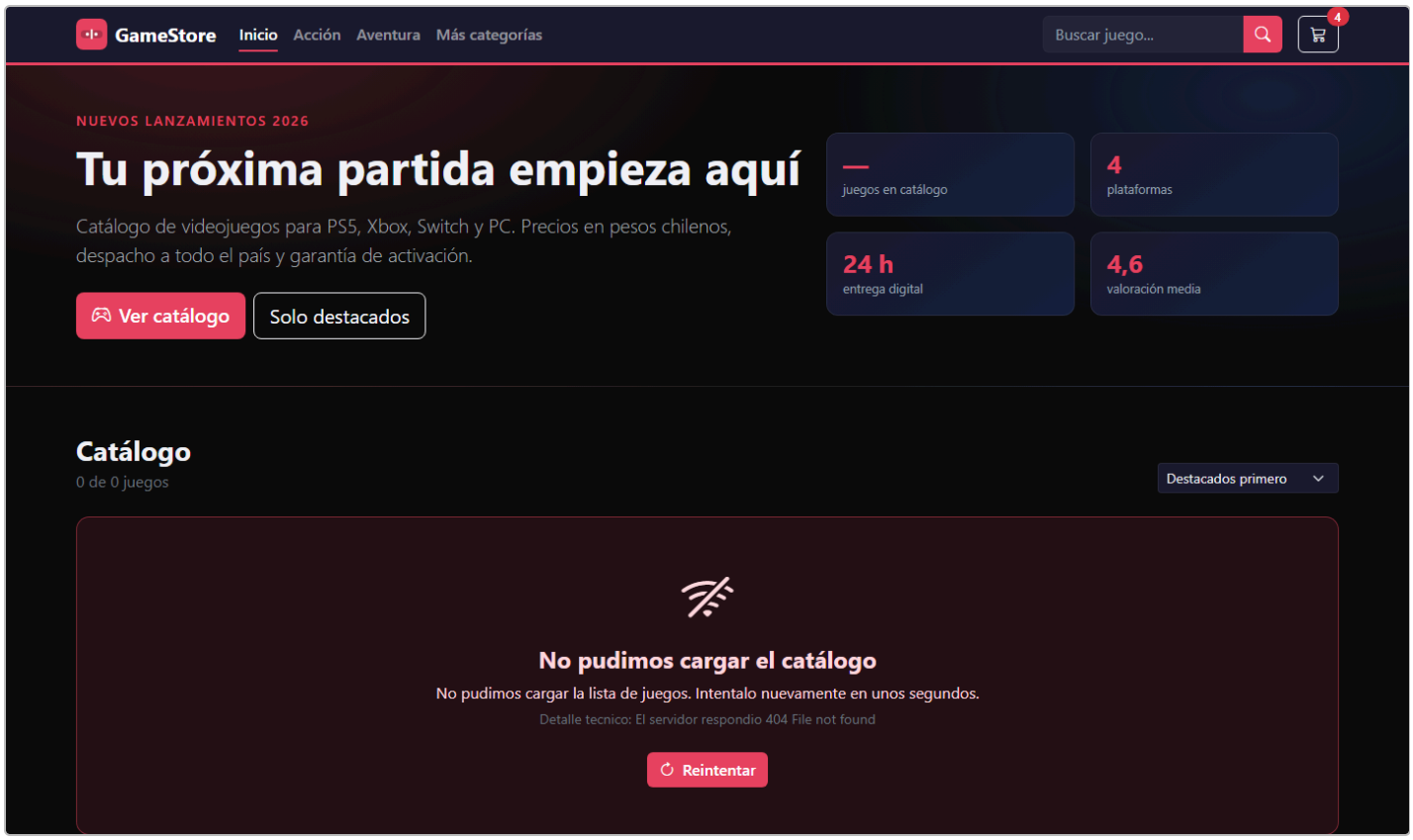
Archivo productos.json servido por el servidor local.



Catálogo ya renderizado: "12 de 12 juegos". El contador de la portada también se llena con el dato del JSON.

Gestión de errores

Si el archivo JSON no se puede cargar, en lugar de quedar la página vacía se muestra un mensaje claro, el detalle del error y un botón para reintentar. La captura se tomó quitando el archivo del servidor a propósito.



Error de carga: mensaje amigable, detalle técnico (404) y botón Reintentar.

Código de la petición

```
const respuesta = await fetch(RUTA_JSON, { cache: 'no-store' });

if (!respuesta.ok) {
    throw new Error(`El servidor respondio ${respuesta.status} ${respuesta.statusText}`);
}

const datos = await respuesta.json();
```

Se revisa **respuesta.ok** porque fetch solo falla por error de red: un 404 o un 500 llegan como respuesta correcta.